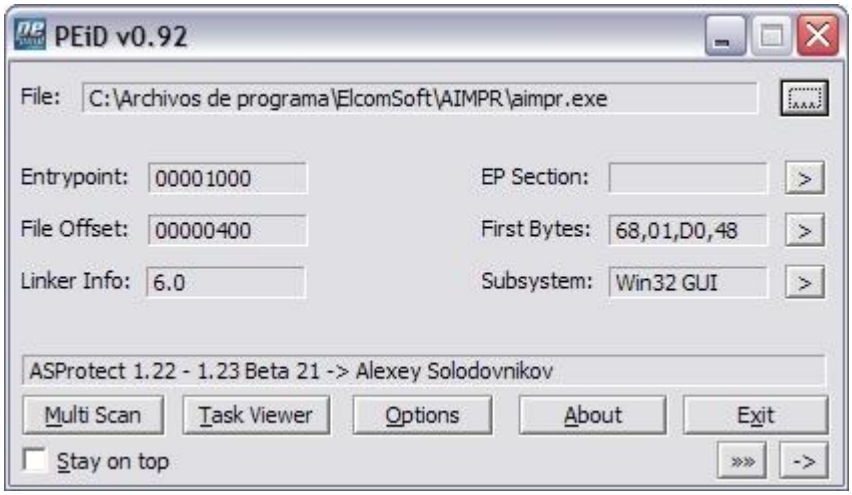
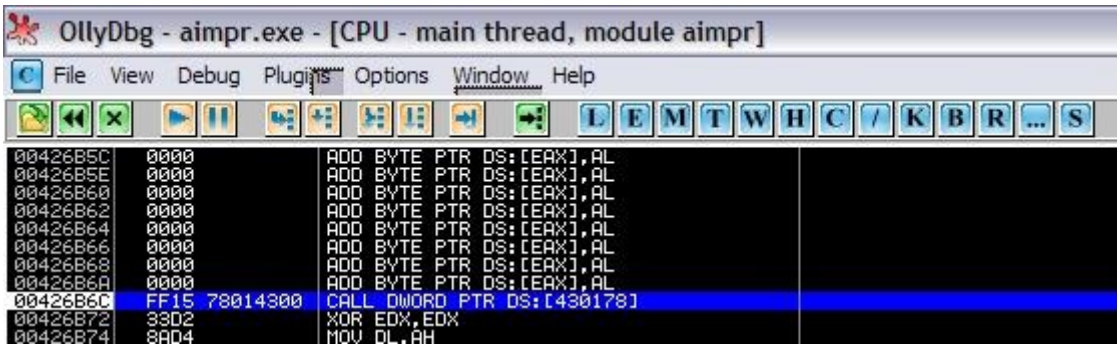
Programa:	Advanced Instant Messengers Password Recovery 2.70	
Protección:	Empaquetado con ASProtect y Funciones deshabilitadas	
Objetivos:	Desempaquetarlo y eliminar todas las restricciones	
URL descarga:	---	
Dificultad:	Newbie avanzado (No es muy difícil)	
Herramientas:	OllyDbg, ImpRec, LordPE	
Cracker:	SyXe'05 [hAcK-c0d3d]	Fecha: 18-07-2005

Conozcamos más al rival de hoy:

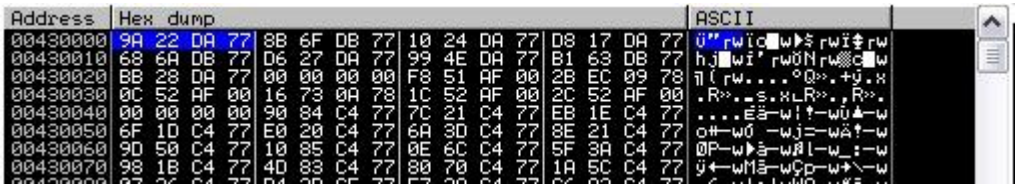


Bueno un viejo Rockero, jeje.. el ASProtect todavía sobrevive? Tantos tutos sobre este packer hacen que esl trabajo sea menos difícil, conocemos de sobra como atacarle, primeramente cargamos el programa en Olly y llegamos hasta la última excepción:

Ahora repetimos el proceso pero al llegar aquí ponemos un BPM On Acces en la primera sección para que pare en el OEP y luego pulsamos Shift+F9, caemos aqui:



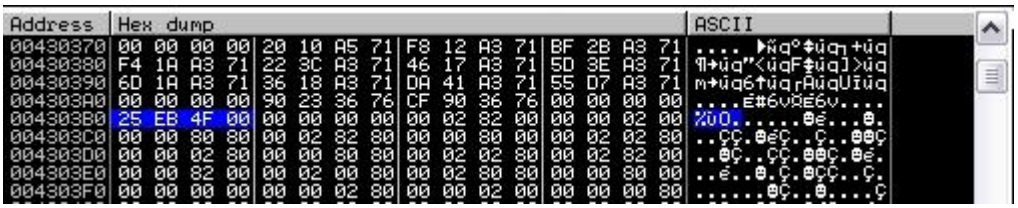
No hace falta que diga que tenemos stolen bytes, todos esos ceros por encima del call son ellos mismos, ahora que estamos aquí determinemos los valores de la IAT que nos harán falta, aquí justo donde estamos en el call nos vale ya que es un salto a la IAT, en este caso es un call pero igual da lo seguimos con “Follow in dump / Memory Address” para caer en plena tabla, desde aquí buscamos el inicio hacia arriba, en este caso es fácil determinar donde empieza porque ya no se puede subir más ya que es el inicio de la sección .idata suponemos claro ya que ASProtect les borra el nombre, que majo oye! Bueno veamos la foto:



El puntero remarcado en azul es nuestro inicio de tabla:

Inicio IAT: 430000

Del mismo modo buscamos el final pero ahora hacia abajo:



Este es nuestro final de tabla:

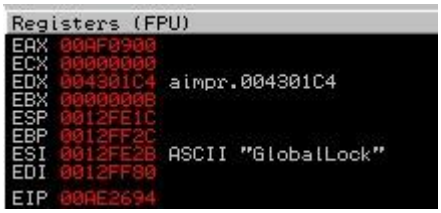
Final IAT: 4303b4

El largo es fácil calcular porque es el final menos el inicio:

Final: 4303b4 – 4300000 = 3b4
Final IAT: 3b4

Como vemos la IAT no es muy grande, com ya tenemos los datos ahora evitaremos que ASProtect redirija muchos punteros ya que ahora mismo habrá no menos de 50 redirigidos (y eso que la IAT es bastante pequeña), lo que haremos será poner un “Breakpoint / Memory On Write” y reiniciar, destildamos la casilla de las excepciones para que Olly no pare constantemente, ahora nos centraremos en ver quién escribe en la tabla, de esta manera estaremos muy cerca del call a nopear, jeje.. tras una paradita extra que novale, damos F9 y paramos aqui:

En este momento está relllenando la IAT, si miramos los registros vemos esto:



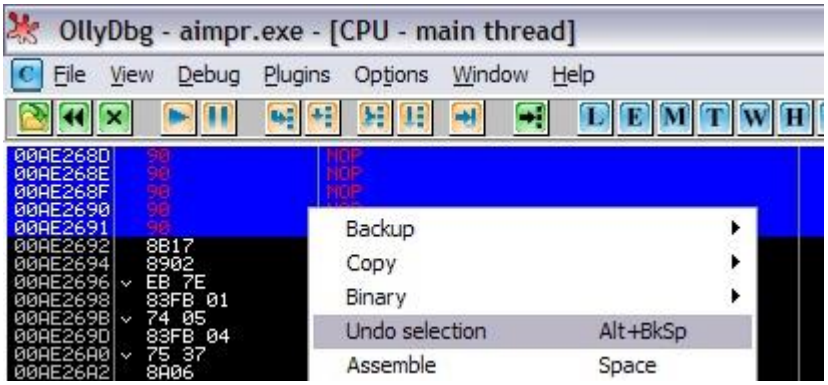
No sé por que no se ve bien la foto porque el archivo original tiene muy buena calidad, este Word!! Bueno se ve como edx vale 4301c4 que es la entrada de la IAT que está tratando en este momento, luego en eax está la API redirigida en este caso y que va a ponoer allí, si ponemos un BPHE en esta direccion y vamos pulsando f9 veremos como va rellenando la tabla con valores buenos y malos, pero lo que más llama la atención es que viene con una API correcta en EAX en cada pasada pero al pasar por el “call 00AE251C” pierde el valor correcto y se

lleva otro hacia direcciones de ASProtect, está claro que este es el call semi-mágico, es más si habéis leído otros tutos de ASProtect no os será difícil identificarlo, ahora que lo tenemos:

Call semi-mágico: 00AE268D

Lo que haremos ahora será reiniciar todo otra vez y nopear el call para evitar que nos redireccione las APIs, primero llegaremos a la primera excecpción pues antes esta zona no estará creada y Olly dirá esto si intentáis ir:

no os olvidéis cuando empieza por una letra poner un cero delante sino olly no lo reconocerá, vale para evitar esto como digo llegar a la primera excepción y desde allí si que lo aceptará, poner un BPX con F2 sobre el call, sale un mensaje decir que si, luego destildar las excepciones para que el Olly no pare, de esto no pongo foto pues no creo que haya problema, y llegar al call con F9, ahora vamos al último puntero de la IAT [4303b0] y ponemos un 'Bp memory on Write' para que se detenga allí al escribir, luego vamos al call y hacemos 'Undo Selection' así:



así dejaremos el call en su lugar y no tendremos problemas con el CRC del archivo, jeje.. no olvidar dejar en su sitio las tildes de las excepciones para que Olly pare, llegar a la última que tenemos anotada, poner el BPM en la primera sección y llegar al OEP, todo este ritual es casi siempre igual en los ASProtect de esta versión (los Aspr2 son otro mundo aunque tampoco varía mucho sólo los antidumps), suponemos que hemos reparado parte de la IAT al nopear el call y estamos en el OEP abrimos ImpRec y seleccionamos el proceso de nuestro programa, en los datos que pide ponemos:

OEP: 00026B6C
RVA: 00030000
Size: 000003B4

Recordad que trabajamos en offsets por eso se le resta el imagebase que es 400000 en este caso (igual para la mayoría de exes) damos a 'Get Imports' y nos dice ImpRec que quedaron 8 punteros inválidos, bueno ya son menos y no son muchos tampoco, hagamos una mini-tablita con estas enTradas a ver:

Entrada:	IAT rva:	Apunta a:
1.-	30144	AE13F0
2.-	30178	AE13C4
3.-	3017C	AE1408
4.-	301F0	AE0F2C
5.-	3022C	AE1430
6.-	30230	AE139C
7.-	3023C	AE1420
8.-	30310	AE144C

Ahora que tenemos el lugar de la Iat donde están los punteros inválidos que debemos reparar, los que haremos será ir a cada una de ellas con el ImpRec y mirar que hay, además el propio Import tiene un plugin para repararlas pero es preferible hacerlo a mano por seguridad, como digo vamos una a una, elegimos la opción'Dissasemble / Hex view' y nos mostrará una vista de la entrada, repasemos las que hay:

Entrada1.

```
DISASSEMBLER / HEX VIEWER

00AE13F0    mov eax,[AE6CEC]    // DWORD value: FFFFFFFF
00AE13F5    retn
```

Esta devuelve eax=FFFFFFFF (eax = -1) y es GetCurrentProcess ¿parece mentira no?

Entrada2.

```
DISASSEMBLER / HEX VIEWER

00AE13C4    push dword ptr [AE6CE8]    // DWORD value: 0A280105
00AE13CA    pop eax
00AE13CB    retn
```

Esta devuelve 05 en EAX que es el resultado de la llamada a GetVersion, lo que hace ASProtect es llamar a esta Api desde su propio codigo y almacenar el resultado en una variable para ahora recuperarlo, quiere despistar, je...

Entrada3.

```
DISASSEMBLER / HEX VIEWER

00AE1408    push 0
00AE140A    call 00AD5160    // = kernel32.dll/016F/GetModuleHandleA
00AE140F    push dword ptr [AE6CE8]    // DWORD value: 0A280105
00AE1415    pop eax
00AE1416    mov eax,[AE6CF8]    // DWORD value: 00151EE0
00AE141C    retn
```

Esta parece que es GetModuleHandleA pero no es cierto porque despues hace un push y un pop eax que recoge el valor del push anterior en eax y despues un mov eax de mas, al final retorna en eax el valor 151ee0 que es un puntero al path del programa como podemos ver yendo hasta alli:

```
Address   Hex dump                                          ASCII
00151EE0  22 43 3A 5C 41 72 63 68 69 76 6F 73 20 64 65 20  "C:\Archivos de
00151EF0  70 72 6F 67 72 61 6D 61 5C 45 6C 63 6F 6D 53 6F  programa\ElcomSo
00151F00  66 74 5C 41 49 4D 50 52 5C 61 69 6D 70 72 2E 65  ft\AIMPR\aimpr.e
00151F10  78 65 22 00 AB AB AB AB AB AB AB AB EE FE EE FE  xe".½½½½½½½½■■■■
00151F20  00 00 00 00 00 00 00 00 5E 00 0A 00 00 07 18 00  ........^.....↑.
00151F30  20 22 15 00 50 25 15 00 00 00 29 00 0C 00 00 00   "§.P%§...).....
00151F40  3A 17 2C 00 00 00 00 00 00 00 00 00 00 00 00 00  :↨,.............
```

Como vemos es el resultado de la llamada a GetCommandLineA que ASProtect trabaja como en GetVersion pues hace exactamente lo mismo, jeje..

Entrada4.

```
DISASSEMBLER / HEX VIEWER

00AE0F2C    push ebp
00AE0F2D    mov ebp,esp
00AE0F2F    mov edx,[ebp+C]
00AE0F32    mov eax,[ebp+8]
00AE0F35    mov ecx,[AE5448]     // DWORD value: 00AE5310
00AE0F3B    mov ecx,[ecx]
00AE0F3D    cmp ecx,eax
00AE0F3F    jnz short 00AE0F4A
00AE0F41    mov eax,[edx*4+AE534C]     // DWORD value: 00000000
00AE0F48    jmp short 00AE0F51
00AE0F4A    push edx
00AE0F4B    push eax
00AE0F4C    call 00AD5168     // = kernel32.dll/0191/GetProcAddress
00AE0F51    pop ebp
00AE0F52    retn 8
```

Esta no necesita comentario alguno, es GetProcAddress sin duda alguna.

Entrada5.

```
DISASSEMBLER / HEX VIEWER

00AE1430    push ebp
00AE1431    mov ebp,esp
00AE1433    pop ebp
00AE1434    retn 4
```

Esta es FreeResource aunque parezca incríble tmbién

Entrada6.

```
DISASSEMBLER / HEX VIEWER

00AE139C    push ebp
00AE139D    mov ebp,esp
00AE139F    mov eax,[ebp+8]
00AE13A2    test eax,eax
00AE13A4    jnz short 00AE13B9
00AE13A6    cmp dword ptr [AE6978],400000
00AE13B0    jnz short 00AE13B9
00AE13B2    mov eax,[AE6978]     // DWORD value: 00400000
00AE13B7    jmp short 00AE13BF
00AE13B9    push eax
00AE13BA    call 00AD5160     // = kernel32.dll/016F/GetModuleHandleA
00AE13BF    pop ebp
00AE13C0    retn 4
```

Esta tampoco deja lugar a dudas GetModuleHandleA.

Entrada7.

```
DISASSEMBLER / HEX VIEWER

00AE1420    push ebp
00AE1421    mov ebp,esp
00AE1423    mov eax,[AE6CF8]     // DWORD value: 00151EE0
00AE1429    mov eax,[ebp+8]
00AE142C    pop ebp
00AE142D    retn 4
```

Esta guarda un valor en eax que resulta ser un puntero al path del programa pero poco despues lo chafa con otro mov eax asi que no nos dejemos engañar, esta es LockResource para S.O. Win9x y SetHandleCount para WinNT, lo mas comodo es dejarla como LockResource y el propio S.O. se encarga de cargarla como SetHandleCount si procede pues de repararla asi no correria bajo Win9x, total es LockResource.

Entrada8.

```
DISASSEMBLER / HEX VIEWER

00AE144C    push ebp
00AE144D    mov ebp,esp
00AE144F    push ebx
00AE1450    mov ebx,[ebp+8]
00AE1453    mov eax,[ebp+18]
00AE1456    push eax
00AE1457    mov eax,[ebp+14]
00AE145A    push eax
00AE145B    mov eax,[ebp+10]
00AE145E    push eax
00AE145F    push 5
00AE1461    mov eax,[ebp+C]
00AE1464    push eax
00AE1465    push ebx
00AE1466    call 00AD5108    // = kernel32.dll/00D9/FindResourceA
00AE146B    push eax
00AE146C    push ebx
00AE146D    call 00AD5190    // = kernel32.dll/0240/LoadResource
00AE1472    push eax
00AE1473    call 00AD5198    // = kernel32.dll/024E/LockResource
00AE1478    push eax
00AE1479    push ebx
00AE147A    call 00AD51B8    // = user32.dll/009C/DialogBoxIndirectParamA
00AE147F    pop ebx
00AE1480    pop ebp
00AE1481    retn 14
```

Esta aunque puede asustar a primera vista no es gran cosa, açun con todo lo que he llegado a ver verdaderas peripecias para repararla en algunos tutoriales he visto hasta injertos en una nueva seccion, pero señores esta se resuelve como DialogBoxParamA, el mismo plugin del Import la repara asi igual que el Stripper.

Como conclusion nos debera haber quedado un cuadro asi:

Entrada:	IAT rva:	Apunta a:	API correcta:
1.-	30144	AE13F0	GetCurrentProcess
2.-	30178	AE13C4	GetVersion
3.-	3017C	AE1408	GetCommandLineA
4.-	301F0	AE0F2C	GetProcAddress
5.-	3022C	AE1430	FreeResource
6.-	30230	AE139C	GetModuleHandleA
7.-	3023C	AE1420	LockResource
8.-	30310	AE144C	DialogBoxParamA

Estas son todas las entradas invalidas de la IAT, ahora a por los stolen bytes, jeje.. ya queda menos, como primer paso nos situamos justo sobre la ultima excepcion (disculpar si no los pongo la tecla de acento va cuando queire), bien miremos en el stack a ver donde retornaremos cuando superemos la excepcion, miremos donde triene ASProtect instalao el manejador de excepciones (SE handler).

Ver foto siguiente pag.

```
0012FF60  0012FFE0  Pointer to next SEH record
0012FF64  00AE2D5D  SE handler
0012FF68  00AD0000
0012FF6C  003E0000
0012FF70  00AE33A8  ASCII "UÏÿÏ┤U«"
0012FF74  00AF9E34  ASCII "8ƒ↓"
0012FF78  00AF3470
0012FF7C  00400000  ASCII "MZé"
0012FF80  0048D7D5  aimpr.0048D7D5
```

Ahí vemos que tiene la SEH instalada en :AE2D5D, debemos poner un BPM on acces en la seccion donde este esta instalada, vamos a la ventana de memoria con Alt+M a ver:

```
006B0000 00002000                          Map  R E   R E
006C0000 00103000                          Map  R     R
007D0000 000B0000                          Map  R E   R E
00AD0000 0001D000                          Priv RW    RW
00AF0000 0000C000                          Priv RW
00BF0000 00051000                          Map  R     R
5B150000 00001000 uxtheme          PE header Imag R     RWE
5B151000 0002C000 uxtheme  .text   code,import Imag R     RWE
5B17D000 00001000 uxtheme  .data   data      Imag R     RWE
5B17E000 00004000 uxtheme  .rsrc   resources Imag R     RWE
5B182000 00002000 uxtheme  .reloc  relocations Imag R     RWE
```

En la seccion :AD0000-AED000 (largo seccion: 1D000) ponemos el BPM y pasamos la excepcion con Shift+f9

```
OllyDbg - aimpr.exe - [CPU - main thread]
File View Debug Plugins Options Window Help
00AE2D5D   8B4424 0C      MOV EAX,DWORD PTR SS:[ESP+C]
00AE2D61   8380 B8000000  ADD DWORD PTR DS:[EAX+B8],2
00AE2D68   C740 18 000000 MOV DWORD PTR DS:[EAX+18],0
00AE2D6F   31C0           XOR EAX,EAX
00AE2D71   C3             RETN
00AE2D72   31C0           XOR EAX,EAX
```

Aquí salimos al superar la excepcion y con el BPM puesto, si llegamos al ret y lo pasamos caeremos en ntdll.dll pero no importa F9 y retornamos al codigo de ASProtect, seguimos hasta el siguiente ret con F8 y ya mismo podemos comenzar la traza para trincar los stolen, tened presente que hay varios bucles para molestar nuestra labor, jeje pero no importa porque los superamos com bpx al final y f9 quitamos el bpx y seguimos, tampoco pasemos ningun call con f8 porque saldra el programa en pantalla, es momento de configurar la traza, en “Debug / Set Condition” ponemos esto:

```
Condition to pause run trace
Pause run trace when any checked condition is met:
[ ] EIP is in range          00000000 ... 00000000
[ ] EIP is outside the range 00000000 ... 00000000
[x] Condition is TRUE        EAX==12ffe0
```

para que pare cuando EAX tome el valor 12FFE0 que es el qu tenia en el falso OEP, por esto tuvimos que anotar este valor, ahora se entiende verdad? Entonces picamos en el pequeñito boton de los puntos suspensivos y seleccionamos “Log to File” con el boton derecho del raton si no hicieramos esto es posible que no vieramos nada ya que ASProtect borra sus pasos antes de saltar al OEP y de esta manera aunque esto pase tendremos todo bien guardado en el archivo que queramos nosotros, jeje.. luego comenzamos la traza con Ctrl+F11, en Olly veremos:

```
Top  Tracing
```

que indica que esta traceando, de vez en cuando mirar en el log picamos en el boton de los puntos suspensivos otra vez y miremos que no estemos en uno de esos loops, si estamos ir a “Debug / Open or clear run trace” y cerramos la traza, cerramos tambien el archivo que donde estamos guardando todo con “Close File” salimos del bucle y reiniciamos el proceso para que no haya mucha paja en el archivo resultante, en este caso no hace falta hacer nada de esto porque rapido para aquí:

```
OllyDbg - aimpr.exe - [CPU - main thread]
File View Debug Plugins Options Window Help

00AF3D0E   8D8D C37F4900   LEA ECX,DWORD PTR SS:[EBP+497FC3]
00AF3D14   8901            MOV DWORD PTR DS:[ECX],EAX
00AF3D16   55              PUSH EBP
00AF3D17   8BEC            MOV EBP,ESP
00AF3D19   6A FF           PUSH -1
00AF3D1B   68 00154300     PUSH 431500
00AF3D20   68 F4C14200     PUSH 42C1F4
00AF3D25   64:A1 00000000  MOV EAX,DWORD PTR FS:[0]
00AF3D2B   50              PUSH EAX
00AF3D2C   64:8925 000000  MOV DWORD PTR FS:[0],ESP
00AF3D33   83EC 58         SUB ESP,58
00AF3D36   53              PUSH EBX
00AF3D37   56              PUSH ESI
00AF3D38   57              PUSH EDI
00AF3D39   8965 E8         MOV DWORD PTR SS:[EBP-18],ESP
00AF3D3C   68 AD3CAF00     PUSH 0AF3CAD                    ASCII "'£³┐├<»"
00AF3D41   C3              RETN
00AF3D42   90              NOP
```

Ahí se detuvo Olly porque eax tomo el valor perseguido 12FFE0 y estamos justamente donde se ejecutan los Stolen Bytes, jeje.. que son estos:

Push ebp
Mov ebp, esp
Push -1
Push 431500
Push 42c1f4
Mov eax, dword ptr FS:[0]
Push eax
Mov dword ptr FS:[0], esp
Sub esp, 58
Push ebx
Push esi
Push edi
Mov dword ptr [ebp-18], esp

Luego ya viene un push que no forma parte de los stolen asi que son estos, típicos de un programa compilado en C++ pues son siempre muy similares, ahora que tenemos nuestros bytes vamos al dumpeado y los agragamos, tambien podemos llegar al OEP desde aquí con F8 hay que tracear poco y ponerlos, los copiamos con “Binary Copy” y los añadimos seleccionando los ceros antes del call y con “Binary Paste” al final debe quedar algo asi:

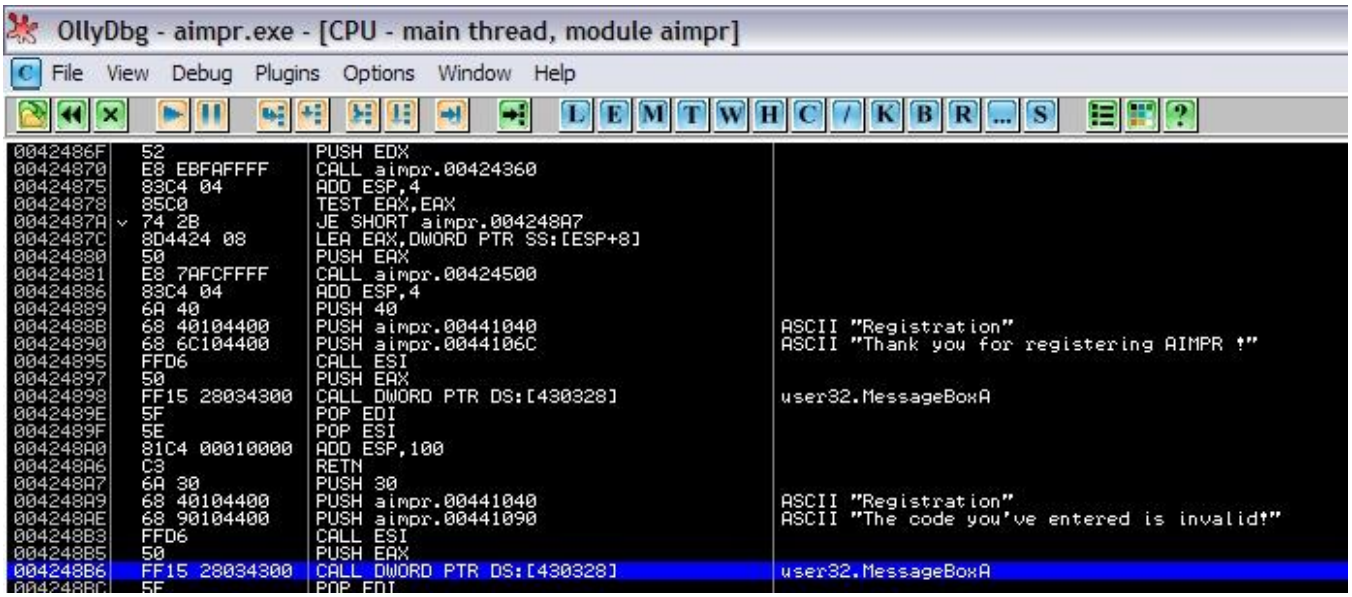
```
OllyDbg - Dump.exe - [CPU - main thread, module Dump]
File View Debug Plugins Options Window Help

00426B46  $ 55             PUSH EBP
00426B47  . 8BEC           MOV EBP,ESP
00426B49  . 6A FF          PUSH -1
00426B4B  . 68 00154300    PUSH Dump.00431500
00426B50  . 68 F4C14200    PUSH Dump.0042C1F4              SE handler installation
00426B55  . 64:A1 000000   MOV EAX,DWORD PTR FS:[0]
00426B5B  . 50             PUSH EAX
00426B5C  . 64:8925 0000   MOV DWORD PTR FS:[0],ESP
00426B63  . 83EC 58        SUB ESP,58
00426B66  . 53             PUSH EBX
00426B67  . 56             PUSH ESI
00426B68  . 57             PUSH EDI
00426B69  . 8965 E8        MOV DWORD PTR SS:[EBP-18],ESP
00426B6C  . FF15 78B14B0   CALL DWORD PTR DS:[<&kernel32.GetVersio  kernel32.GetVersion
00426B72  . 33D2           XOR EDX,EDX
00426B74  . 8AD4           MOV DL,AH
```

Este ya tiene la IAT reparada por eso se ve el call a GetVersion, bueno, ahora probemos el arhivo resultante, no olvidar de cuadrar el OEP para que apunte a :426B46 y no donde estaba sino no correra el archivo dumpeado, cuando tengamos todo esto hecho (el archivo volcado con la IAT reparada, los stolen bytes agragados y el OEP epuntando a :426B46) ya podemos probarlo, lo lanzamos y… FUNCIONA..!! a la primera aranco que raro en estos tiempos casi siempre se guardan alguna sorpresa mas anti-dumpeado y cosas asi.. ahora que tenemos el programa desempaquetado podemos preocuparnos por las limitaciones… en este tema no ahondare mucho primero porque no he probado el prorgama y no se que restricciones tiene, solo he visto que tiene la opcion de registro y por aquí atacaremos, vamos a hacer que acepte cualquier serial si no podemos encontrar uno valido, como no hay que poner nombre sino que es un serial solo sabemos que este debera pasar ciertas restricciones, etc.,

en estos casos no suele ser facil sacar un serial valido pero veamos, primeramente carguemos el programa dumpeado y reparado en Olly y lo dejamos correr, ingresamos un serial y…

Esta sera nuestra puerta de entrada, busquemos el mensaje entre las strings del programa a ver donde aparecemos al hacer doble clic sobre ella con el raton en caso de encontrarla, pues si esta y salimos aquí:



Aqui estan ambos mensaje: el de felicitacion y el de error poco mas abajo, arriba de estos hay un salto que determina que mensaje se mostrara pero mas importante es el call que controla el salto (supuestamente la rutina de verificacion del serial) pongamos un bp en el call e introduzcamos otro serial, cuando Olly salte entremos con F7 y traceamos el call por dentro, yo lo he hecho y es una locura de las operaciones que le hacen al serial, total que opte por parchear la rutina para que retornara eax=1 y crea que el serial es valido sin serlo, jeje.. no esta nada mal tampoco esta opcion, entonces lo que hay que hacer es obligarlo a devolver eax=1 como hay varios situos por donde puede retornar ponemos todas las salidas con eax=1 y punto, pero antes esto:

:00424376 83F9 06 CMP ECX,6
:00424379 7D 07 JGE SHORT aimpr.00424382

aqui esta mirando si el serial ingresado supera los 6 digitos, para no tener problemas con esto jumpeamos el salto, o sea cambiaremos:

:00424379 7D 07 JGE SHORT aimpr.00424382 **POR**:
:00424379 EB 07 JMP SHORT aimpr.00424382

Y seguimos a lo nuestro tapando salidas, los siguientes cambios seran:

:00424393 33C0 XOR EAX,EAX **POR**:
:00424393 B0 01 MOV AL,1

:004243ED 33C0 XOR EAX,EAX **POR**:
:004243ED B0 01 MOV AL,1

00424434 75 07 JNZ SHORT aimpr.0042443D **POR**:
00424434 EB 07 JMP SHORT aimpr.0042443D

Con esto aseguramos que el call retorne eax=1 como queremos, jeje.. seleccionamos cualquier cambio y elegimos la opcion “Copy to executable / All modifications” saldra un mensaje donde pulsaremos en “Copy All” luego se abrira otra ventana donde selecccionaremos finalmente “Save File” y le damos un nombre, probamos el archivo resultante y…

jeje.. el boton de registro quedo deshabilitado lo que indica que el parche parece haber funcionado bien, el resumen de todo el tema explicado es este:

Ultima excepcion: AE2D7A
OEP: 426b6c Hay stolen… eax vale =12ffe0
IAT: Inicio: 430000
Final: 4303b4
Largo: 3b4
Call semimagico: 00AE268D

Para las limitaciones del programa:

```
00424870  |.  E8 EBFAFFFF    CALL Dump_.00424360    ;debe retornar eax=1
00424875  |.  83C4 04        ADD ESP,4
00424878  |.  85C0           TEST EAX,EAX
0042487A  |.  74 2B          JE SHORT Dump_.004248A7
```

y ya esta por hoy, tenemos el programa libre de ASProtect y de limitaciones, ya digo yo no lo he probado si alguien lo prueba y le da problemas porfavor que me lo haga saber que revisare lo que sea necesario hasta que quede el crack funcional.

Y como siempre decir que si alguien ademas tiene alguna duda, sugerencia, critica por que no tambien acerca de este texto u otro estcrito por mi que me escriba a cualquiera de estas dos direcciones:

syxe00@yahoo.es
syxe00@hotmail.com

Responderé a todo el que escriba, eso seguro.